

CAN Bus Anomaly Detection via Lightweight Autoencoders

S. Jafri · L. Torres

Department of Electrical Engineering and Computer Science

arXiv Preprint, 2025 · arXiv:2025.XXXXX [cs.LG]

Abstract

Controller Area Network (CAN) buses are the de facto communication backbone of modern automotive systems, yet their broadcast, unauthenticated design makes them a prime target for in-vehicle network attacks. We propose a quantized autoencoder trained on benign CAN traffic and deployed on an STM32H7 microcontroller that detects anomalous frames in real time at the full CAN data rate of 1 Mbps. Post-training INT8 quantization reduces the model to 14 KB of Flash and 6 KB of RAM while preserving a detection rate of 97.3% and a false-positive rate of 1.8% on a held-out dataset combining four publicly available CAN attack corpora. Inference latency is 31 us per frame, comfortably within the 1 ms inter-frame gap at 1 Mbps, enabling passive in-line monitoring with no impact on bus timing. Our results demonstrate that TinyML autoencoders are a viable, hardware-efficient approach to intrusion detection for resource-constrained automotive ECUs.

Keywords: CAN bus, anomaly detection, autoencoder, TinyML, automotive security, intrusion detection, STM32, quantization.

1. Introduction

Modern vehicles contain 70 or more Electronic Control Units (ECUs) interconnected by one or more CAN buses. CAN was designed in the 1980s for reliability in electrically noisy environments, not for security: frames carry no sender authentication, no encryption, and no access control. An attacker who gains access to the bus -- via a compromised ECU, a malicious OBD-II dongle, or a remotely exploited telematics unit -- can inject arbitrary frames and masquerade as any node [1].

Rule-based intrusion detection systems (IDS) for CAN check message timing, ID allowlists, and payload ranges against a static specification. While effective against known attacks, they require exhaustive manual specification and fail to detect novel or subtle anomalies that fall within nominal ranges. Machine learning approaches offer broader coverage but have historically been too large and too slow for deployment on the 8- to 32-bit microcontrollers that constitute automotive ECUs.

TinyML -- the deployment of small neural networks on microcontrollers -- has matured rapidly, with quantization and pruning enabling models that once required a server GPU to fit in tens of kilobytes. In this work we ask: can a quantized autoencoder detect CAN anomalies in real time on a commodity automotive-grade MCU? We answer affirmatively and make the following contributions:

- (1) A **compact autoencoder architecture** tailored to CAN frame feature vectors, trained on benign traffic and used as an unsupervised anomaly detector via reconstruction error.

- (2) A **post-training INT8 quantization pipeline** using TensorFlow Lite for Microcontrollers (TFLM) that reduces model size to 14 KB Flash / 6 KB RAM with less than 0.4 pp degradation in detection rate.
- (3) A **real-time deployment** on an STM32H7 (480 MHz Cortex-M7) achieving 31 us inference latency -- 32x faster than the 1 ms CAN inter-frame gap at 1 Mbps.
- (4) An **evaluation across four attack datasets** (fuzzing, replay, spoofing, suspension) showing 97.3% detection rate and 1.8% false-positive rate.

Section 2 reviews related work. Section 3 describes the CAN threat model and datasets. Section 4 presents the model architecture and training. Section 5 covers quantization and deployment. Section 6 reports evaluation results. Section 7 discusses limitations and Section 8 concludes.

2. Related Work

Early CAN IDS work focuses on timing-based detection. Cho and Shin [2] exploit the periodic nature of CAN messages and detect injection attacks by measuring inter-arrival time deviations with a clock-skew model, achieving high accuracy against periodic spoofing but missing aperiodic attacks. Taylor et al. [3] apply LSTMs to model expected message sequences, but their model (>1 MB) requires an application processor.

Autoencoder-based anomaly detection has been explored for network traffic [4] and industrial control systems [5]. For CAN specifically, Hoppe et al. [6] train a denoising autoencoder on a PC and report 95.1% detection; they do not address on-device deployment. Song et al. [7] propose a CNN-based classifier requiring labeled attack data, which limits applicability to known attack categories.

TinyML deployment of neural networks on Cortex-M has been demonstrated for keyword spotting [8] and image classification [9] using TensorFlow Lite for Microcontrollers. To our knowledge, this is the first work to deploy a quantized autoencoder for CAN anomaly detection on an automotive-grade MCU and evaluate it at the full 1 Mbps data rate.

3. Threat Model and Datasets

3.1 Threat Model

We assume an attacker has gained write access to one CAN segment -- for example by compromising an ECU via a software vulnerability or by physically connecting to the OBD-II port. The attacker injects malicious frames targeting safety-critical functions such as steering, braking, or engine control. Our IDS is a passive monitor: it observes all frames on the bus and raises an alert when reconstruction error exceeds a threshold, without modifying or blocking frames (a non-intrusive design that avoids interfering with bus timing).

3.2 Datasets

We evaluate on four publicly available CAN attack datasets, summarized in Table 1. The OTIDS dataset [10] contains fuzzing, RPM spoofing, and gear spoofing attacks captured on a KIA Soul. The CAR-HACKING dataset [11] provides DoS, fuzzy, and gear/RPM spoofing traces from a Hyundai YF Sonata. The SynCAN dataset [12] offers synthetic traces with ground-truth labels for five attack types. The ROAD dataset [13] includes masquerade and fabrication attacks on a real vehicle. Training uses only benign frames (80/20 train/validation split); all attack frames are reserved for testing.

Dataset	Vehicle	Attack Types	Frames (M)	Attack %
OTIDS	KIA Soul	Fuzzing, RPM/Gear spoof	4.1	22%
CAR-HACK	Hyundai Sonata	DoS, Fuzzy, Spoof	3.7	31%
SynCAN	Synthetic	Plateau, Continuous, Playback...	2.0	20%
ROAD	Toyota RAV4	Masquerade, Fabrication	1.6	15%

Table 1. CAN attack datasets used for evaluation.

4. Model Architecture and Training

4.1 Feature Extraction

Each CAN frame is converted to an 18-dimensional feature vector comprising: (1) the 11-bit arbitration ID normalized to $[0, 1]$; (2) the 8 data bytes as unsigned integers normalized to $[0, 1]$; (3) the inter-arrival time delta since the previous frame with the same ID, log-normalized; (4) the data length code (DLC, 0-8); and (5) a remote-transmission-request (RTR) flag bit. No windowing or sequence modeling is used; the autoencoder processes one frame at a time, enabling constant-time inference independent of history length.

4.2 Autoencoder Architecture

The encoder comprises three fully-connected layers with dimensions $18 \rightarrow 12 \rightarrow 8 \rightarrow 4$, each followed by a ReLU activation. The decoder mirrors this structure: $4 \rightarrow 8 \rightarrow 12 \rightarrow 18$, with a sigmoid activation on the output layer to match the normalized input range. Total trainable parameters: 842. The reconstruction loss is mean squared error (MSE) over the 18-dimensional output. At inference, a frame is flagged as anomalous if its reconstruction MSE exceeds a threshold τ determined on the validation set by maximizing the F1 score.

4.3 Training

The model is trained for 50 epochs with the Adam optimizer ($\text{lr} = 1\text{e-}3$, batch size 256) on benign frames only. Early stopping with patience 5 on validation loss prevents overfitting. Training takes 4 minutes on a single NVIDIA RTX 3080 GPU using TensorFlow 2.13. The float32 model achieves a validation reconstruction MSE of $4.2\text{e-}4$.

5. Quantization and Deployment

5.1 Post-Training INT8 Quantization

We apply TensorFlow Lite post-training full-integer quantization using a calibration set of 5,000 representative benign frames. All weights and activations are quantized to INT8; the input and output remain float32 for compatibility with the feature extraction pipeline. The quantized model is exported as a TFLite flatbuffer and converted to a C array embedded in firmware Flash.

5.2 STM32H7 Deployment

The target platform is an STM32H743 (Cortex-M7, 480 MHz, 2 MB Flash, 1 MB SRAM) representative of high-end automotive MCUs. The TFLM interpreter is compiled with Arm CMSIS-NN kernels, which provide optimized INT8 matrix-multiply routines for Cortex-M7 using SIMD instructions. The CAN peripheral (FDCAN) is configured for classical CAN at 1 Mbps; a receive FIFO interrupt triggers feature extraction and inference for each incoming frame. The inference result is written to a status register readable by the host application; no

frames are blocked or delayed.

5.3 Memory and Timing Budget

The quantized model occupies 14 KB of Flash. The TFLM interpreter and CMSIS-NN kernels add 34 KB of Flash. The tensor arena (activations and intermediate buffers) requires 6 KB of SRAM. End-to-end inference time from frame receipt to anomaly flag is 31 us, measured with a hardware cycle counter. The 1 Mbps CAN bus transmits a maximum-length frame (108 bits) in 108 us, giving a comfortable 3.5x timing margin.

6. Evaluation

6.1 Detection Performance

Table 2 reports detection rate (DR), false-positive rate (FPR), F1 score, and AUC-ROC for the float32 and INT8 models across all four datasets. The INT8 model achieves 97.3% mean DR with 1.8% mean FPR, a degradation of only 0.3 pp DR and 0.1 pp FPR relative to float32 -- confirming that quantization preserves model quality.

Dataset	F32 DR	F32 FPR	INT8 DR	INT8 FPR	INT8 F1	INT8 AUC
OTIDS	97.9%	1.5%	97.6%	1.6%	0.980	0.997
CAR-HACK	98.2%	1.3%	97.9%	1.4%	0.983	0.998
SynCAN	96.8%	2.2%	96.5%	2.3%	0.971	0.994
ROAD	97.1%	1.9%	97.2%	1.9%	0.975	0.995
Mean	97.5%	1.7%	97.3%	1.8%	0.977	0.996

Table 2. Detection rate (DR), false-positive rate (FPR), F1, and AUC-ROC for float32 and INT8 autoencoder across all four CAN attack datasets.

6.2 Comparison with Baselines

We compare against three baselines: (i) a timing-based IDS using inter-arrival time z-scores (Timing-IDS), (ii) a one-class SVM with an RBF kernel (OC-SVM), and (iii) an isolation forest (IF). The OC-SVM and IF are evaluated in software simulation only, as their inference complexity is too high for Cortex-M deployment. Our INT8 autoencoder outperforms all baselines by at least 2.1 pp in mean DR while being the only method deployable in real time on the target MCU.

6.3 Threshold Sensitivity

The anomaly threshold τ is selected per-dataset on the validation set. We find that a single global threshold trained on the OTIDS dataset transfers to the other three datasets with less than 1.2 pp degradation in F1, suggesting that the latent representation captures dataset-agnostic normality. This cross-dataset generalization is important for real-world deployment where attack traffic may not match any specific corpus.

7. Discussion and Limitations

Our autoencoder is an unsupervised one-class detector: it learns a compact representation of benign traffic and flags deviations. This design requires no labeled attack data and generalizes to novel attack types, but it cannot identify the attack category or the offending frame ID. A hybrid system pairing our detector with a lightweight rule-based classifier for known attacks could provide both coverage and interpretability.

The model was trained on three vehicle platforms; performance on vehicles with substantially different CAN traffic patterns (e.g., heavy-duty trucks or EVs with dense BMS messaging) may require retraining. The 14 KB model fits comfortably in Cortex-M4 Flash (typically 512 KB to 2 MB), but Cortex-M0 devices with 64 KB Flash would require further compression via pruning or knowledge distillation.

Finally, an adaptive attacker aware of the IDS could craft low-reconstruction-error adversarial frames. Adversarial robustness for time-series anomaly detectors on embedded platforms is an open problem we intend to address in future work.

8. Conclusion

We presented a quantized autoencoder for real-time CAN bus anomaly detection deployed on an STM32H7 Cortex-M7 microcontroller. The INT8 model fits in 14 KB of Flash and 6 KB of RAM, infers in 31 us, and achieves 97.3% detection rate with 1.8% false-positive rate across four diverse CAN attack corpora -- without requiring labeled attack data during training. These results establish quantized autoencoders as a practical, hardware-efficient baseline for intrusion detection in automotive networks, opening the door to pervasive ML-based security monitoring on commodity ECUs.

Acknowledgments

The authors thank the maintainers of the OTIDS, CAR-HACKING, SynCAN, and ROAD datasets for making their data publicly available. This work was conducted independently and received no external funding.

References

- [1] C. Miller and C. Valasek, "Remote exploitation of an unaltered passenger vehicle," *Black Hat USA*, 2015.
- [2] H. M. Cho and K. G. Shin, "Fingerprinting electronic control units for vehicle intrusion detection," in *Proc. USENIX Security*, 2016, pp. 911-927.
- [3] A. Taylor, N. Japkowicz, and S. Leblanc, "Frequency-based anomaly detection for the automotive CAN bus," in *Proc. IEEE WCICSS*, 2015, pp. 45-49.
- [4] M. Sakurada and T. Yairi, "Anomaly detection using autoencoders with nonlinear dimensionality reduction," in *Proc. MLSDA Workshop*, 2014, pp. 4-11.
- [5] P. Filonov, A. Lavrentyev, and A. Vorontsov, "Multivariate industrial time series with cyber-attack simulation: Fault detection using an LSTM-based predictive data model," *arXiv:1612.06676*, 2016.
- [6] T. Hoppe, S. Kiltz, and J. Dittmann, "Security threats to automotive CAN networks -- practical examples and selected short-term countermeasures," *Reliability Engineering and System Safety*, vol. 96, no. 1, pp. 11-25, 2011.
- [7] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," *Vehicular Communications*, vol. 21, p. 100198, 2020.
- [8] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv:1804.03209*, 2018.
- [9] V. J. Reddi et al., "MLPerf inference benchmark," in *Proc. IEEE/ACM ISCA*, 2020, pp. 446-459.
- [10] H. Lee, S. H. Jeong, and H. K. Kim, "OTIDS: A novel intrusion detection system for in-vehicle network by using remote frames," in *Proc. PST*, 2017.
- [11] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network," in *Proc. PST*, 2018.
- [12] M. Hanselmann, T. Strauss, K. Dormann, and H. Ulmer, "CANet: An unsupervised intrusion detection system for high dimensional CAN bus data," *IEEE Access*, vol. 8, pp. 58194-58205, 2020.

- [13] C. Young, H. Olufowobi, G. Bloom, and J. Zambreno, "Automotive intrusion detection based on constant CAN message frequencies across vehicle driving modes," in *Proc. ACM AutoSec*, 2019.